

Activation Functions in Deep Learning — Complete Notes

Compiled Study Notes

2026

Contents

Introduction	6
1. Linear Activation Function	7
Explanation	7
Math	7
Graph	7
Code	7
How to use	8
Where it's used	8
Why used	8
Disadvantages	8
2. Binary Step Function	9
Explanation	9
Math	9
Graph	9
Code	9
How to use	10
Where it's used	10
Why used	10
Disadvantages	10
3. Sigmoid (Logistic) Function	11
Explanation	11
Math	11
Graph	11
Code	11
How to use	12
Where it's used	12
Why used	12
Disadvantages	12
4. Tanh (Hyperbolic Tangent)	13
Explanation	13
Math	13

Graph	13
Code	13
How to use	14
Where it's used	14
Why used	14
Disadvantages	14
5. ReLU (Rectified Linear Unit)	15
Explanation	15
Math	15
Graph	15
Code	15
How to use	16
Where it's used	16
Why used	16
Disadvantages	16
6. Leaky ReLU	17
Explanation	17
Math	17
Graph	17
Code	17
How to use	18
Where it's used	18
Why used	18
Disadvantages	18
7. Parametric ReLU (PReLU)	19
Explanation	19
Math	19
Graph	19
Code	19
How to use	20
Where it's used	20
Why used	20
Disadvantages	20
8. ELU (Exponential Linear Unit)	21
Explanation	21
Math	21
Graph	21
Code	21
How to use	22
Where it's used	22
Why used	22
Disadvantages	22
9. SELU (Scaled Exponential Linear Unit)	23

Explanation	23
Math	23
Graph	23
Code	23
How to use	24
Where it's used	24
Why used	24
Disadvantages	24
10. Softmax	25
Explanation	25
Math	25
Graph	25
Code	25
How to use	26
Where it's used	26
Why used	26
Disadvantages	26
11. Softplus	27
Explanation	27
Math	27
Graph	27
Code	27
How to use	28
Where it's used	28
Why used	28
Disadvantages	28
12. Softsign	29
Explanation	29
Math	29
Graph	29
Code	29
How to use	30
Where it's used	30
Why used	30
Disadvantages	30
13. Swish (SiLU)	31
Explanation	31
Math	31
Graph	31
Code	31
How to use	32
Where it's used	32
Why used	32
Disadvantages	32

14. Mish	33
Explanation	33
Math	33
Graph	33
Code	33
How to use	34
Where it's used	34
Why used	34
Disadvantages	34
15. GELU (Gaussian Error Linear Unit)	35
Explanation	35
Math	35
Graph	35
Code	35
How to use	36
Where it's used	36
Why used	36
Disadvantages	36
16. Hard Sigmoid	37
Explanation	37
Math	37
Graph	37
Code	37
How to use	38
Where it's used	38
Why used	38
Disadvantages	38
17. Hard Tanh	39
Explanation	39
Math	39
Graph	39
Code	39
How to use	40
Where it's used	40
Why used	40
Disadvantages	40
18. Maxout	41
Explanation	41
Math	41
Graph	41
Code	41
How to use	42
Where it's used	42
Why used	42

Disadvantages	42
19. ReLU6	43
Explanation	43
Math	43
Graph	43
Code	43
How to use	44
Where it's used	44
Why used	44
Disadvantages	44
20. Master Comparison Table	45
21. How to Choose an Activation Function — Practical Guide	47
General rules of thumb	47
22. The Vanishing & Exploding Gradient Problem (Why Activation Choice Matters So Much)	48
23. Implementing a Tiny Neural Network From Scratch (Putting It Together)	49
24. Summary Cheat Sheet (One-Liners)	51
25. References & Further Reading	52

Introduction

Activation functions are mathematical functions applied to the output of a neuron (a weighted sum of inputs plus bias) in a neural network. They decide **whether a neuron should “fire”** and **how strongly**, and most importantly, they introduce **non-linearity** into the network.

Why do we need them at all?

If we removed activation functions completely, a neural network — no matter how many layers it had — would collapse mathematically into a single linear transformation:

$$y = W_n(W_{n-1}(\dots (W_1x + b_1) \dots) + b_{n-1}) + b_n = W'x + b'$$

A purely linear model can only learn linear decision boundaries. It cannot learn things like XOR, image recognition, language patterns, or anything with curved/complex decision boundaries. Activation functions are what give deep learning its **representational power**.

What this document covers, for every function:

- Plain-language explanation
 - Mathematical formula and derivative
 - Graph (input vs output)
 - Python code (NumPy / PyTorch)
 - How to use it in practice
 - Where it is used (real architectures)
 - Why it is used (advantages)
 - Disadvantages / when to avoid it
 - Quick comparison notes
-

1. Linear Activation Function

Explanation

The Linear (Identity) activation function simply returns the input as the output, scaled by a constant. It does **not** add any non-linearity — what goes in, comes out unchanged (or proportionally scaled).

Math

$$f(x) = x$$

Derivative:

$$f'(x) = 1$$

Graph

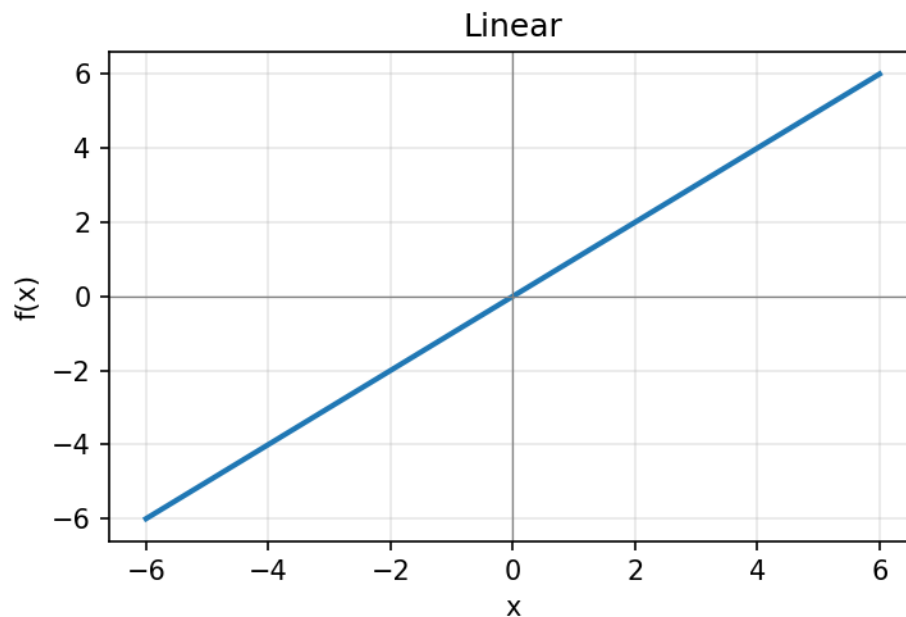


Figure 1: Linear Activation

Code

```
import numpy as np

def linear(x):
    return x

# PyTorch
import torch
import torch.nn as nn
```

```
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
y = x # Identity, nn.Identity() module also available
print(nn.Identity()(x))
```

How to use

Used as the activation of the **output layer** in regression problems, where you want the raw real-valued prediction (e.g., predicting house price, temperature).

```
model = nn.Sequential(
    nn.Linear(10, 64), nn.ReLU(),
    nn.Linear(64, 1) # No activation here = linear output for regression
)
```

Where it's used

- Output layer of regression networks
- Autoencoders' bottleneck/output layer (sometimes)
- Skip/residual connections (implicitly linear)

Why used

- When the predicted value can be any real number (not bounded between 0–1 or -1–1)
- Keeps gradient constant (=1), avoiding vanishing gradient **at that single layer**

Disadvantages

- Cannot be used in hidden layers of deep networks — stacking linear layers gives no extra power over a single layer (no non-linearity)
 - Constant derivative => no real “learning shaping” — gradient descent cannot create complex curves with it alone
 - Cannot solve non-linearly separable problems (e.g., XOR)
-

2. Binary Step Function

Explanation

The simplest activation conceptually — inspired by a biological neuron either “firing” or “not firing.” If the input crosses a threshold (usually 0), output is 1, otherwise 0.

Math

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Derivative: $f'(x) = 0$ everywhere except at $x = 0$, where it's undefined (a discontinuity / Dirac delta).

Graph

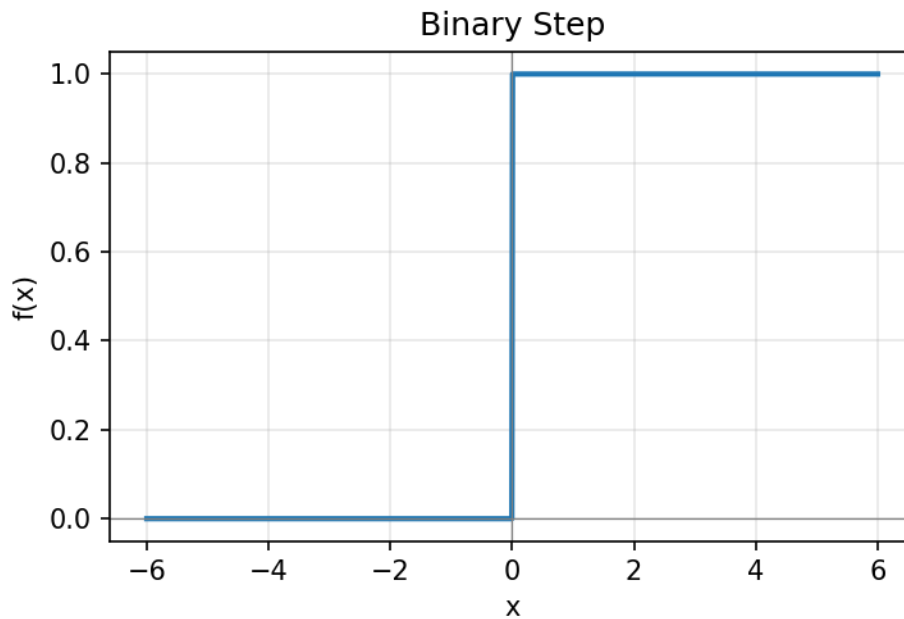


Figure 2: Binary Step

Code

```
import numpy as np

def binary_step(x):
    return np.where(x >= 0, 1, 0)

print(binary_step(np.array([-3, -0.5, 0, 0.5, 3])))
# [0 0 1 1 1]
```

How to use

Rarely used in modern deep learning training pipelines because it isn't differentiable in a useful way, but conceptually used for:

```
# Simple perceptron-style binary classifier (historical / educational)  
def perceptron_output(x, w, b):  
    z = np.dot(x, w) + b  
    return binary_step(z)
```

Where it's used

- Early **Perceptron** models (1950s–60s, McCulloch-Pitts neuron)
- Simple binary classifiers / threshold logic gates
- Spiking neural networks (conceptually, for “spike” generation)

Why used

- Extremely simple, intuitive “on/off” interpretation
- Good for teaching the concept of a neuron firing

Disadvantages

- Gradient is zero almost everywhere => **cannot be used with backpropagation/gradient descent**
 - Cannot output probabilities or any intermediate value
 - Cannot handle multi-class problems
 - Not used in any real deep network today
-

3. Sigmoid (Logistic) Function

Explanation

Sigmoid squashes any real-valued number into the range (0, 1). It looks like an “S”-shaped curve. This makes it historically very popular because outputs can be interpreted as probabilities.

Math

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative (a very useful property — expressed in terms of itself):

$$f'(x) = \sigma(x) (1 - \sigma(x))$$

Graph

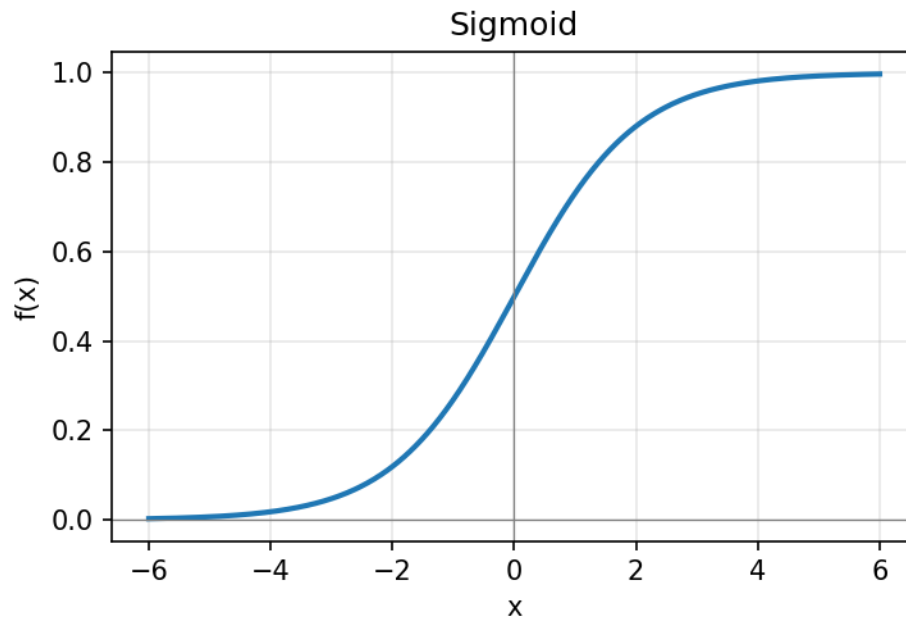


Figure 3: Sigmoid

Code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)
```

```
# PyTorch
import torch
x = torch.tensor([-2.0, 0.0, 2.0])
y = torch.sigmoid(x)
# or
y = torch.nn.Sigmoid()(x)
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Linear(20, 1),
    nn.Sigmoid() # squashes to (0,1) for binary classification probability
)
loss_fn = nn.BCELoss()
```

Where it's used

- Output layer of **binary classification** networks (predicting probability of class 1)
- Gates in **LSTM** / **GRU** recurrent networks (forget gate, input gate, output gate)
- Logistic Regression (the original use case)

Why used

- Smooth, differentiable everywhere
- Interpretable as probability
- Bounded output stabilizes activations

Disadvantages

- **Vanishing gradient**: for large $|x|$, derivative $\rightarrow 0$, so gradients become tiny in deep networks, halting learning in early layers
 - **Not zero-centered**: outputs always positive (0 to 1), which causes zig-zagging gradient updates in deeper layers
 - Computationally a bit more expensive than ReLU (uses `exp`)
-

4. Tanh (Hyperbolic Tangent)

Explanation

Tanh is like a rescaled sigmoid — it squashes inputs into the range $(-1, 1)$ and is **zero-centered**, which usually makes optimization easier than plain sigmoid.

Math

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

Derivative:

$$f'(x) = 1 - \tanh^2(x)$$

Graph

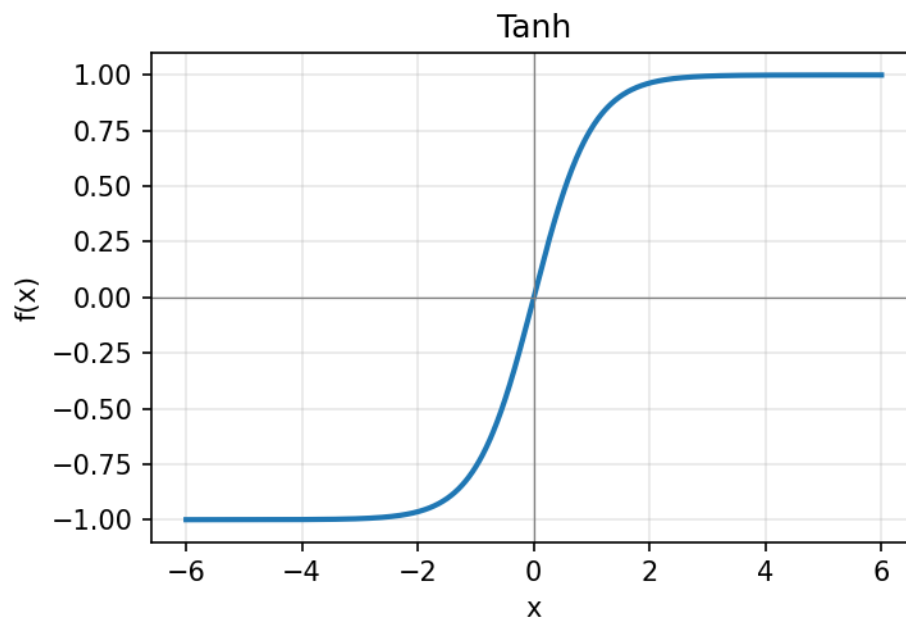


Figure 4: Tanh

Code

```
import numpy as np

def tanh(x):
    return np.tanh(x)

def tanh_derivative(x):
    return 1 - np.tanh(x)**2

# PyTorch
```

```
import torch
y = torch.tanh(torch.tensor([-2.0, 0.0, 2.0]))
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.Tanh(),
    nn.Linear(64, 1)
)
```

Where it's used

- Hidden layers of **RNNs** / **LSTMs** (candidate cell state, hidden state activation)
- Older feed-forward and CNN hidden layers (pre-ReLU era)
- GAN generator output layer (when output is normalized to $[-1, 1]$, e.g. image pixels)

Why used

- Zero-centered output $\rightarrow$ gradients/updates are more balanced than sigmoid
- Stronger gradients near 0 than sigmoid (steeper curve)

Disadvantages

- Still suffers from **vanishing gradient** for large $|x|$
 - Computationally more expensive than ReLU
 - Saturates at both ends just like sigmoid
-

5. ReLU (Rectified Linear Unit)

Explanation

ReLU is the most widely-used activation function in modern deep learning. It outputs the input directly if positive, and zero otherwise. Simple, fast, and avoids the vanishing-gradient problem (for positive inputs).

Math

$$f(x) = \max(0, x)$$

Derivative:

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \\ \text{undefined} & x = 0 \text{ (usually set to 0 or 1)} \end{cases}$$

Graph

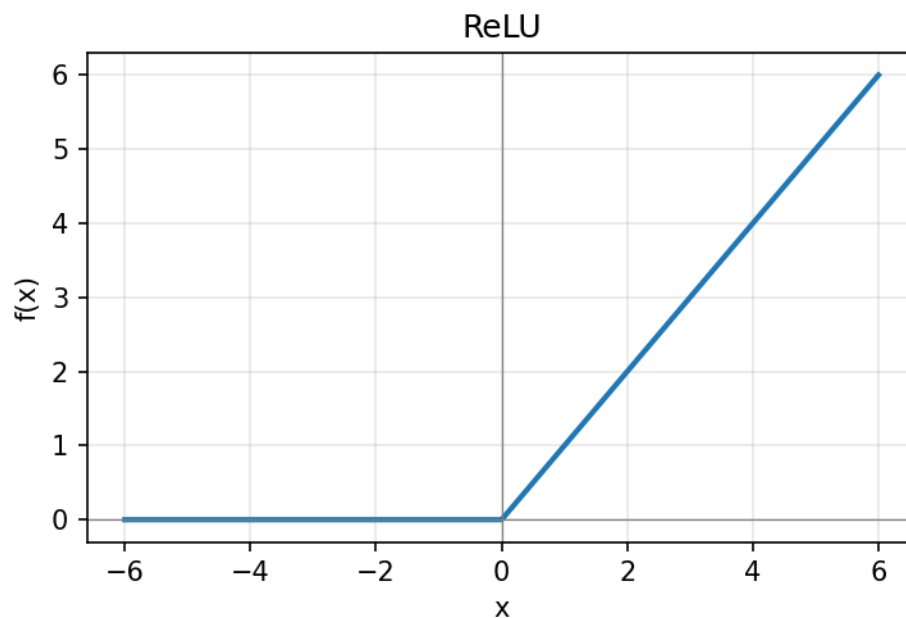


Figure 5: ReLU

Code

```
import numpy as np

def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
```

```

    return np.where(x > 0, 1, 0)

# PyTorch
import torch
import torch.nn as nn
y = torch.relu(torch.tensor([-2.0, 0.0, 3.0]))
# or
y = nn.ReLU()(torch.tensor([-2.0, 0.0, 3.0]))

```

How to use

```

import torch.nn as nn
model = nn.Sequential(
    nn.Conv2d(3, 32, 3), nn.ReLU(),
    nn.Conv2d(32, 64, 3), nn.ReLU(),
    nn.Flatten(),
    nn.Linear(64*6*6, 10)
)

```

Where it's used

- The default activation in almost all **CNNs** (VGG, ResNet hidden layers), feed-forward layers in Transformers (original Transformer FFN uses ReLU), most hidden layers in MLPs

Why used

- Computationally extremely cheap (just a max operation, no exponentials)
- Doesn't saturate for positive inputs => mitigates vanishing gradients
- Induces **sparsity** (many neurons output exactly 0), which can be beneficial for efficiency and representation

Disadvantages

- **Dying ReLU problem:** if a neuron's input is always negative, its gradient is always 0, and it can "die" — never updating again
 - Not zero-centered
 - Unbounded output can occasionally cause exploding activations
-

6. Leaky ReLU

Explanation

A small fix to the Dying ReLU problem. Instead of being exactly zero for negative inputs, Leaky ReLU allows a small, non-zero, constant-sloped output.

Math

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}, \quad \text{typically } \alpha = 0.01$$

Derivative:

$$f'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x < 0 \end{cases}$$

Graph

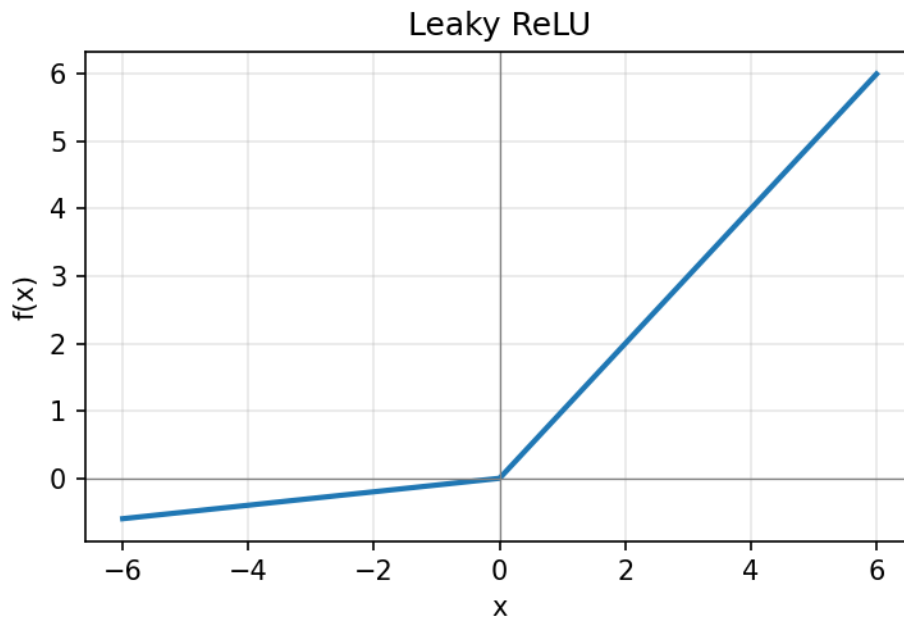


Figure 6: Leaky ReLU

Code

```
import numpy as np

def leaky_relu(x, alpha=0.01):
    return np.where(x > 0, x, alpha * x)

# PyTorch
import torch.nn as nn
```

```
layer = nn.LeakyReLU(negative_slope=0.01)
y = layer(torch.tensor([-2.0, 0.0, 3.0]))
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.LeakyReLU(0.01),
    nn.Linear(64, 1)
)
```

Where it's used

- GANs (very common in **DCGAN discriminators**, where dying neurons are especially harmful)
- CNNs as a drop-in ReLU replacement when dying-neuron problems are observed

Why used

- Keeps a small gradient flowing even for negative inputs => neurons don't "die" completely
- Almost as cheap to compute as ReLU

Disadvantages

- The slope α is a fixed hyperparameter — not learned, may not be optimal
 - Results are inconsistent across tasks — sometimes no better than plain ReLU
 - Still unbounded, not zero-centered
-

7. Parametric ReLU (PReLU)

Explanation

PReLU generalizes Leaky ReLU by making the negative slope α a **learnable parameter**, optimized via backpropagation along with the network weights, rather than a fixed hyperparameter.

Math

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}, \quad \alpha \text{ is learned per-channel or globally}$$

Graph

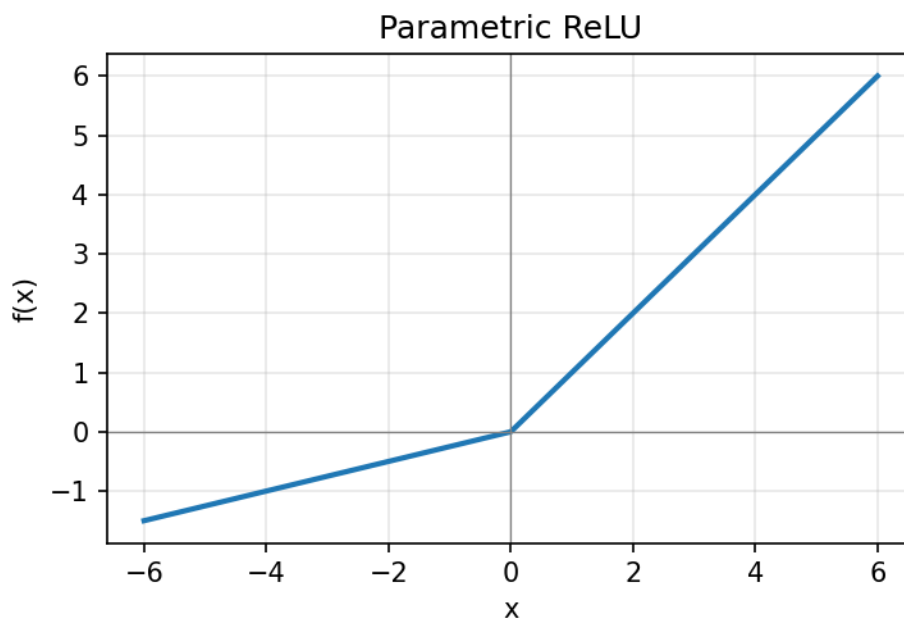


Figure 7: PReLU

Code

```
import torch
import torch.nn as nn

# alpha is a learnable parameter (one per channel by default)
layer = nn.PReLU(num_parameters=1, init=0.25)
x = torch.tensor([-2.0, 0.0, 3.0])
y = layer(x)
print(layer.weight)  # the learned alpha
```

How to use

```
model = nn.Sequential(  
    nn.Conv2d(3, 64, 3),  
    nn.PReLU(num_parameters=64), # one alpha per output channel  
    nn.Conv2d(64, 128, 3),  
    nn.PReLU(num_parameters=128)  
)
```

Where it's used

- Introduced and used in **PReLU-Net** / **Kaiming He's research (2015)** for ImageNet classification
- Used in some face-recognition CNNs (e.g., early ArcFace/SphereFace backbones)

Why used

- Removes the need to manually tune α — the network learns the best leakiness per neuron/channel
- Can outperform ReLU/Leaky ReLU with negligible added parameters

Disadvantages

- Slightly more parameters and computation than ReLU/Leaky ReLU
 - Risk of overfitting on small datasets due to extra learnable parameters
 - Less commonly available/used compared to plain ReLU in lightweight models
-

8. ELU (Exponential Linear Unit)

Explanation

ELU behaves like ReLU for positive values, but for negative values it uses a smooth exponential curve that saturates to $-\alpha$, instead of a hard zero or straight line. This keeps mean activations closer to zero.

Math

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

Derivative:

$$f'(x) = \begin{cases} 1 & x > 0 \\ f(x) + \alpha & x \leq 0 \end{cases}$$

Graph

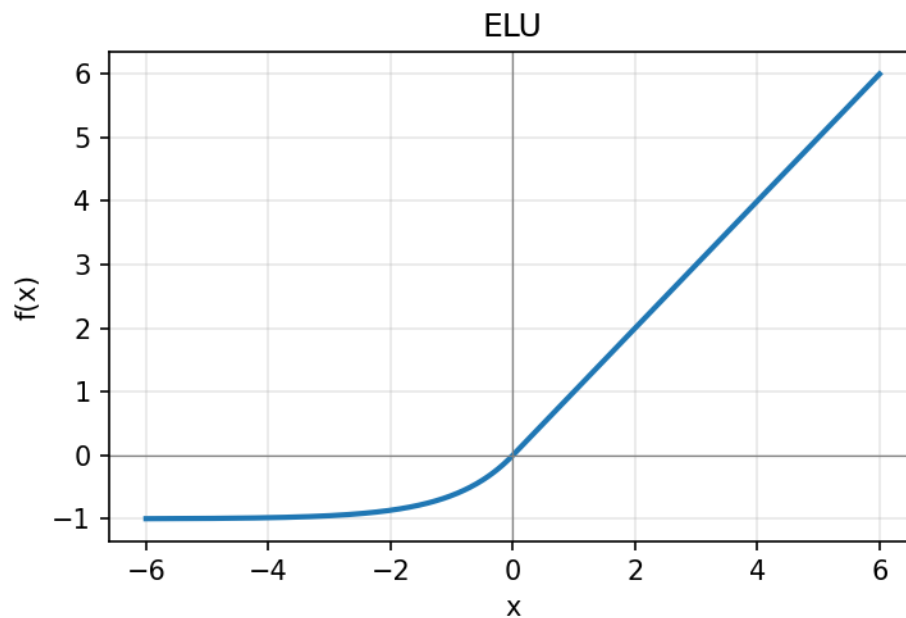


Figure 8: ELU

Code

```
import numpy as np

def elu(x, alpha=1.0):
    return np.where(x > 0, x, alpha * (np.exp(x) - 1))

# PyTorch
```

```
import torch.nn as nn
layer = nn.ELU(alpha=1.0)
y = layer(torch.tensor([-2.0, 0.0, 3.0]))
```

How to use

```
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.ELU(alpha=1.0),
    nn.Linear(64, 1)
)
```

Where it's used

- Some CNN architectures aiming for faster convergence than ReLU
- Used in research comparing smooth activations vs ReLU variants

Why used

- Negative saturation value pushes mean activations closer to zero (similar benefit to batch norm) => faster, more stable learning
- Smooth everywhere (no kink at 0 except in 2nd derivative), can give better gradients than ReLU

Disadvantages

- Computationally more expensive (uses `exp` for negative inputs)
 - Can still saturate for very negative inputs (gradient $\rightarrow 0$ there, although less severe than sigmoid/tanh)
-

9. SELU (Scaled Exponential Linear Unit)

Explanation

SELU is ELU multiplied by a carefully chosen scale factor, designed so that, under certain conditions (specific weight initialization “LeCun normal” and architecture made only of dense layers), the outputs of each layer self-normalize — keeping mean ≈ 0 and variance ≈ 1 automatically across many layers, without needing BatchNorm.

Math

$$f(x) = \lambda \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

with fixed constants (derived mathematically, not arbitrary):

$$\alpha \approx 1.6733, \quad \lambda \approx 1.0507$$

Graph

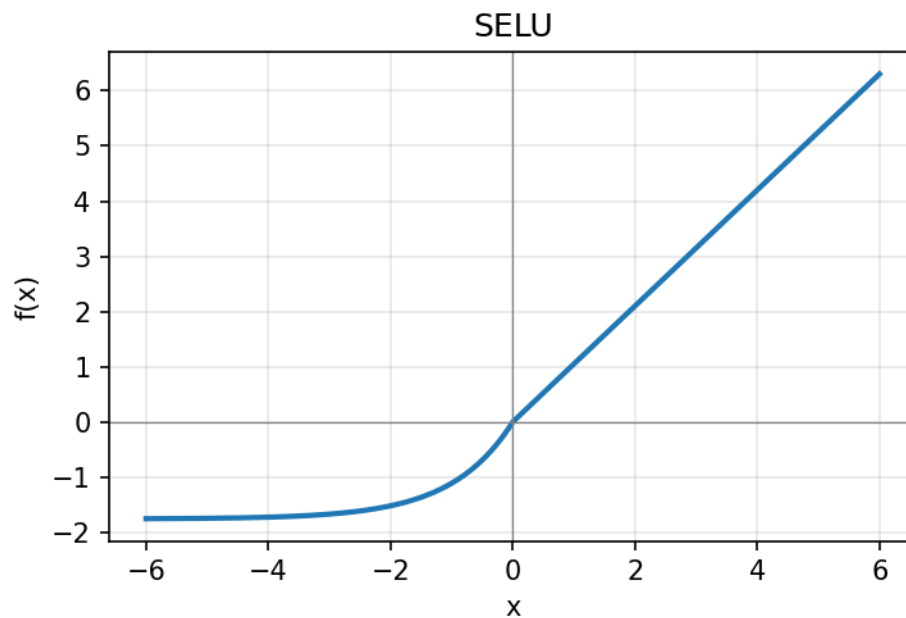


Figure 9: SELU

Code

```
import numpy as np

alpha = 1.6732632423543772848170429916717
scale = 1.0507009873554804934193349852946
```

```
def selu(x):
    return scale * np.where(x > 0, x, alpha * (np.exp(x) - 1))

# PyTorch
import torch.nn as nn
layer = nn.SELU()
y = layer(torch.tensor([-2.0, 0.0, 3.0]))
```

How to use

```
# SELU requires LeCun Normal initialization and AlphaDropout (not regular Dropout)
import torch.nn as nn

class SNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(100, 64)
        self.fc2 = nn.Linear(64, 10)
        self.selu = nn.SELU()
        self.dropout = nn.AlphaDropout(0.1)
        for layer in [self.fc1, self.fc2]:
            nn.init.kaiming_normal_(layer.weight, nonlinearity='linear')

    def forward(self, x):
        x = self.selu(self.fc1(x))
        x = self.dropout(x)
        return self.fc2(x)
```

Where it's used

- **Self-Normalizing Neural Networks (SNNs)** — deep fully-connected networks without BatchNorm
- Tabular data deep learning pipelines where very deep MLPs are used

Why used

- Enables training very deep plain MLPs (e.g., 20+ layers) without BatchNorm/LayerNorm, by automatically maintaining stable activations
- Helps avoid both vanishing and exploding gradients in feed-forward nets

Disadvantages

- Only mathematically guaranteed to self-normalize under specific conditions (fully-connected layers, LeCun normal init, AlphaDropout) — breaks down in CNNs/RNNs or with Batch-Norm
- Less popular/supported than ReLU family in production CNN/Transformer pipelines

10. Softmax

Explanation

Softmax converts a vector of raw scores (“logits”) into a probability distribution — all outputs are between 0 and 1, and they all sum to 1. Unlike the other functions here which act **element-wise**, Softmax acts on an entire **vector** at once (it’s a “joint” activation).

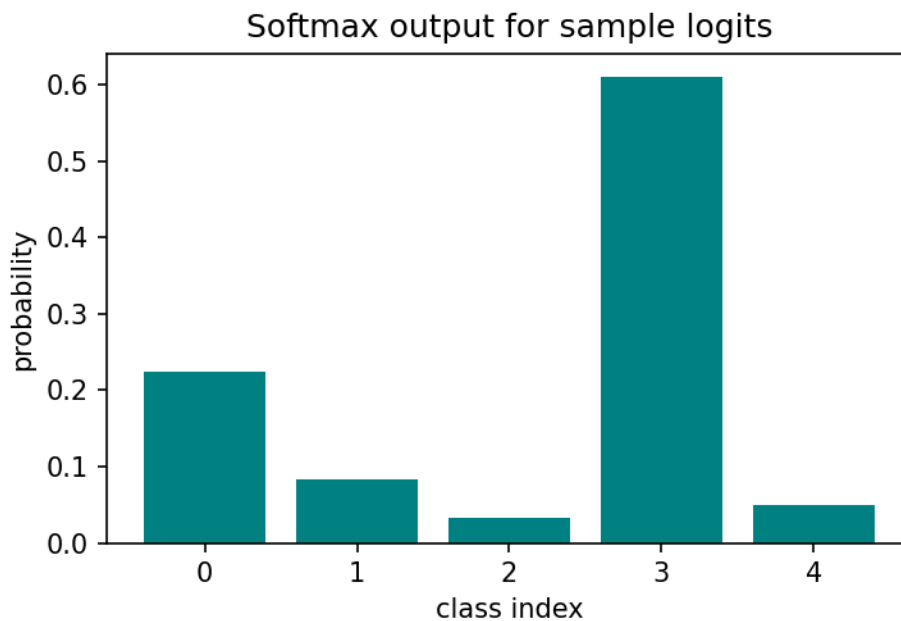
Math

For a vector $z = (z_1, \dots, z_K)$:

$$f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

(In practice we subtract $\max(z)$ before exponentiating, for numerical stability — this doesn’t change the result mathematically.)

Graph



output probabilities for 5 sample logits)

(Bar chart: example

Code

```
import numpy as np

def softmax(z):
    z = z - np.max(z)           # numerical stability trick
    e = np.exp(z)
    return e / np.sum(e)
```

```
logits = np.array([2.0, 1.0, 0.1, 3.0, 0.5])
print(softmax(logits))

# PyTorch
import torch
import torch.nn.functional as F
probs = F.softmax(torch.tensor(logits), dim=-1)
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Linear(128, 10)    # 10-class logits
    # No explicit Softmax layer needed if using nn.CrossEntropyLoss,
    # since it applies log-softmax internally for numerical stability
)
loss_fn = nn.CrossEntropyLoss()

# If you need actual probabilities (e.g. for inference/display):
probs = torch.softmax(model(x), dim=-1)
```

Where it's used

- Output layer of **multi-class classification** networks (image classification, NLP intent classification)
- **Attention mechanism** in Transformers (softmax over attention scores to get attention weights)
- Reinforcement learning policy networks (softmax policy over discrete actions)

Why used

- Produces a valid probability distribution (non-negative, sums to 1) — directly interpretable
- Differentiable, integrates cleanly with cross-entropy loss

Disadvantages

- Sensitive to large logits without the max-subtraction trick (**exp** overflow)
 - Encourages one dominant class (winner-take-most), which can sometimes be overconfident
 - Not suitable for multi-label problems (where multiple classes can be simultaneously true) — use independent sigmoids instead
-

11. Softplus

Explanation

Softplus is a smooth approximation of ReLU. Instead of a hard corner at 0, it transitions smoothly, and importantly, it's **always differentiable everywhere** (unlike ReLU at $x=0$).

Math

$$f(x) = \ln(1 + e^x)$$

Derivative (a clean result — equal to sigmoid!):

$$f'(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

Graph

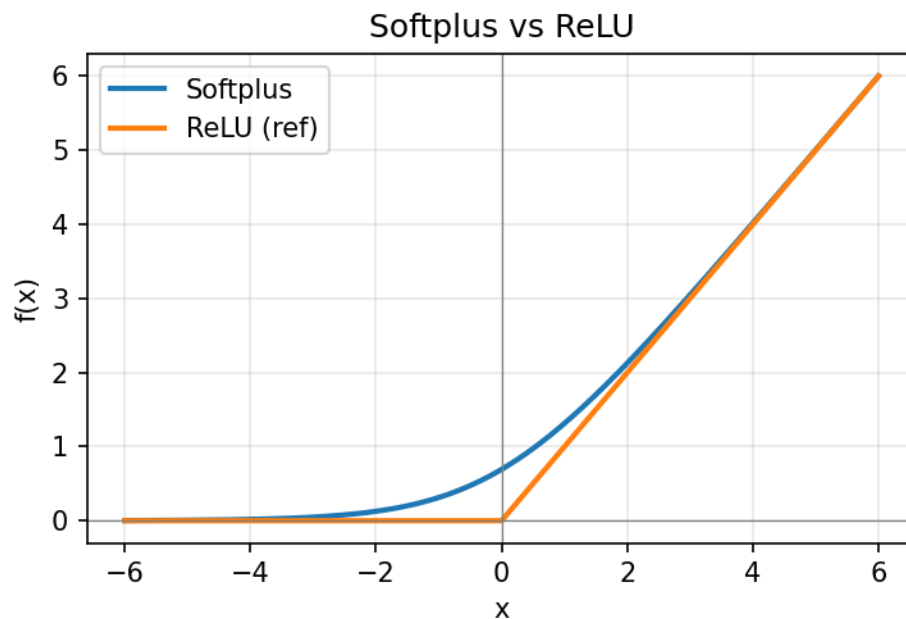


Figure 10: Softplus

Code

```
import numpy as np

def softplus(x):
    # numerically stable version
    return np.log1p(np.exp(-np.abs(x))) + np.maximum(x, 0)

# PyTorch
import torch.nn as nn
y = nn.Softplus()(torch.tensor([-2.0, 0.0, 2.0]))
```

How to use

```
import torch.nn as nn
# Used e.g. to ensure a network output is strictly positive
# (such as predicting a standard deviation in a probabilistic model)
model = nn.Sequential(
    nn.Linear(10, 1),
    nn.Softplus()    # guarantees positive output
)
```

Where it's used

- Predicting **variance/standard deviation** in probabilistic / Bayesian neural networks (must be positive)
- Used inside **Mish** activation (as we'll see) as a building block
- Some reinforcement learning algorithms (e.g., for ensuring positive scale parameters)

Why used

- Smooth, fully differentiable everywhere (no kink), gradients well-behaved
- Naturally enforces strictly positive output, useful where positivity constraints matter

Disadvantages

- Computationally more expensive than ReLU ($\log$, $\exp$)
 - For very negative x , gradient $\rightarrow 0$ (mild vanishing gradient, less severe than sigmoid though)
 - Rarely used directly as a hidden-layer activation in big architectures; mostly a “helper” function
-

12. Softsign

Explanation

Softsign is similar in spirit to Tanh — it squashes inputs to between -1 and 1, but uses a rational (polynomial-based) function instead of exponentials, making it computationally cheaper, with slower-decaying tails.

Math

$$f(x) = \frac{x}{1 + |x|}$$

Derivative:

$$f'(x) = \frac{1}{(1 + |x|)^2}$$

Graph

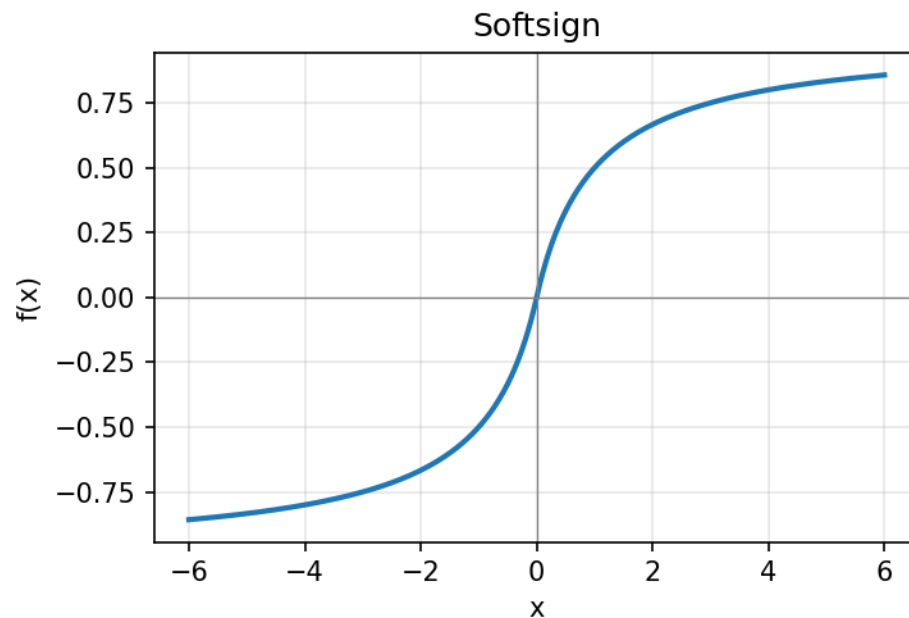


Figure 11: Softsign

Code

```
import numpy as np

def softsign(x):
    return x / (1 + np.abs(x))

# PyTorch
import torch.nn as nn
y = nn.Softsign()(torch.tensor([-2.0, 0.0, 2.0]))
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.Softsign(),
    nn.Linear(64, 1)
)
```

Where it's used

- Occasionally as an alternative to Tanh in RNNs and some shallow networks
- Some older NLP / speech models

Why used

- Cheaper to compute than Tanh (no exponentials, just division)
- Saturates more slowly than Tanh (polynomial decay vs exponential decay), so gradients vanish less aggressively far from zero

Disadvantages

- Still suffers from vanishing gradients for very large $|x|$ (just less severely than Tanh)
 - Far less common / supported than Tanh or ReLU in modern frameworks and pretrained models
 - Generally outperformed by ReLU-family functions in deep networks
-

13. Swish (SiLU)

Explanation

Swish was discovered via automated search (by Google Brain researchers) and found to consistently outperform ReLU on deep networks. It's a “self-gated” function: the input is multiplied by its own sigmoid.

Math

$$f(x) = x \cdot \sigma(\beta x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

When $\beta = 1$, this is also called **SiLU** (Sigmoid Linear Unit):

$$f(x) = x \cdot \sigma(x)$$

Graph

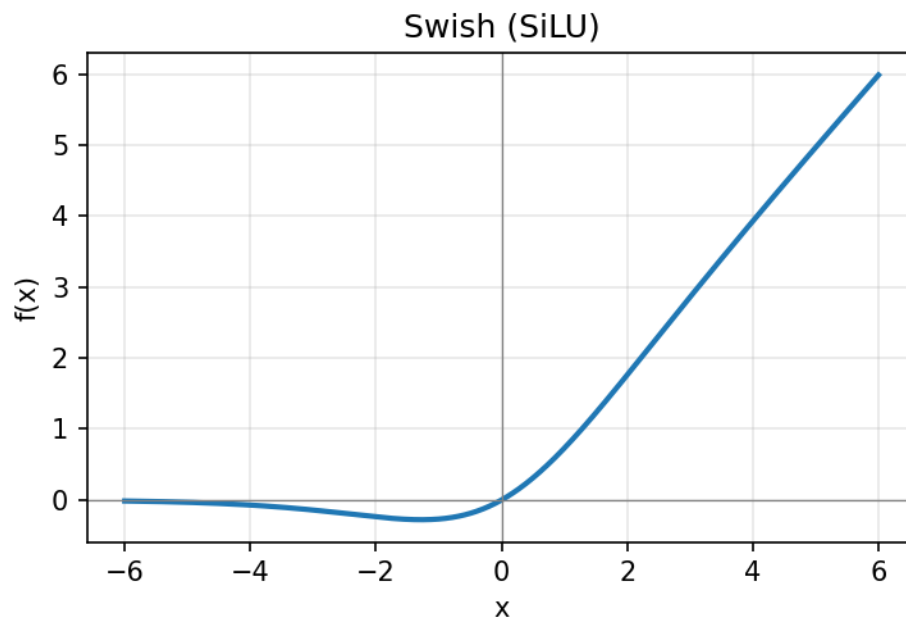


Figure 12: Swish

Code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def swish(x, beta=1.0):
    return x * sigmoid(beta * x)
```

```
# PyTorch (built-in as SiLU when beta=1)
import torch.nn as nn
y = nn.SiLU()(torch.tensor([-2.0, 0.0, 2.0]))
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Conv2d(3, 32, 3),
    nn.SiLU(),           # used heavily in EfficientNet
    nn.Conv2d(32, 64, 3),
    nn.SiLU()
)
```

Where it's used

- **EfficientNet** (Google's image classification family) — core activation function
- **YOLOv5** / **YOLOv7** object detectors
- Some Transformer variants and diffusion model U-Nets (e.g., parts of Stable Diffusion's U-Net use SiLU)

Why used

- Smooth and non-monotonic (dips slightly below 0 near small negative x), found empirically to ease optimization and improve accuracy over ReLU in deep CNNs
- Unbounded above, bounded below — avoids full saturation on the positive side like ReLU, but smoother

Disadvantages

- More expensive to compute than ReLU (sigmoid + multiply)
 - Improvement over ReLU is architecture/task-dependent — not universally better
 - Slightly more complex gradient computation
-

14. Mish

Explanation

Mish is a smooth, non-monotonic activation function similar in spirit to Swish, built using `tanh` and `softplus`. It was designed to provide better gradient flow and was shown to outperform both ReLU and Swish on several benchmarks.

Math

$$f(x) = x \cdot \tanh(\text{softplus}(x)) = x \cdot \tanh(\ln(1 + e^x))$$

Graph

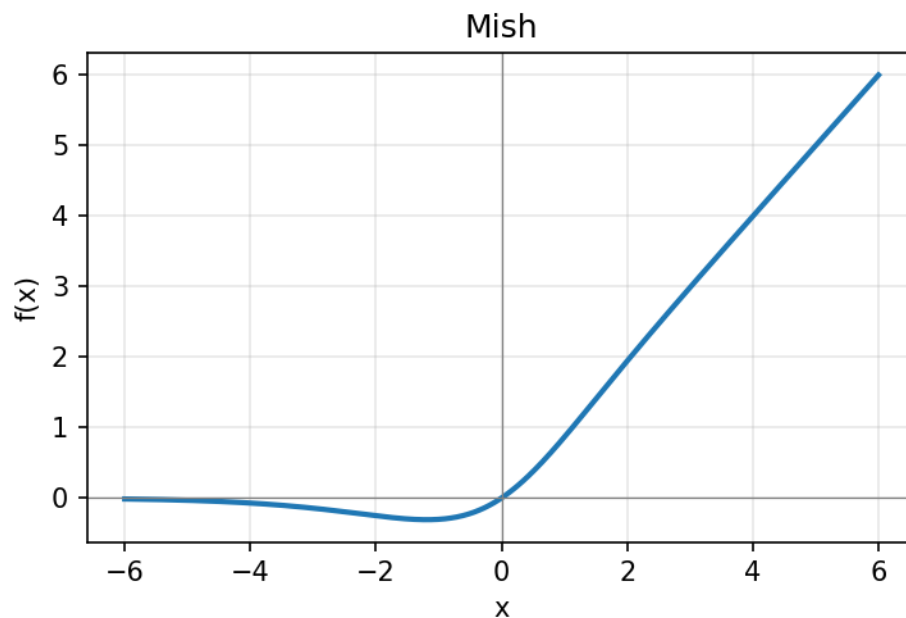


Figure 13: Mish

Code

```
import numpy as np

def softplus(x):
    return np.log1p(np.exp(-np.abs(x))) + np.maximum(x, 0)

def mish(x):
    return x * np.tanh(softplus(x))

# PyTorch
import torch.nn as nn
y = nn.Mish()(torch.tensor([-2.0, 0.0, 2.0]))
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Conv2d(3, 32, 3),
    nn.Mish(),
    nn.Conv2d(32, 64, 3),
    nn.Mish()
)
```

Where it's used

- **YOLOv4** object detector (replaced Leaky ReLU with Mish for a notable accuracy gain)
- Some image classification and segmentation models in research benchmarks

Why used

- Smooth, non-monotonic curve gives better gradient flow than ReLU, reportedly more stable training and improved accuracy in several CNN benchmarks
- Self-regularizing properties (preserves small negative values rather than zeroing them out)

Disadvantages

- More computationally expensive (involves `exp`, `log`, `tanh`)
 - Not as widely supported/pretrained-model-compatible as ReLU/GELU
 - Improvement over Swish/ReLU is often marginal and task-dependent
-

15. GELU (Gaussian Error Linear Unit)

Explanation

GELU weights the input by the value of the standard Gaussian cumulative distribution function (CDF) at that input — intuitively, it’s like a “soft,” probabilistic version of ReLU’s gating: instead of a hard 0/1 gate, the gate value depends smoothly on how likely a Gaussian random variable would be $\leq x$.

Math

$$f(x) = x \cdot \Phi(x), \quad \Phi(x) = P(X \leq x), \quad X \sim \mathcal{N}(0, 1)$$

A commonly-used fast approximation (the one most libraries actually compute):

$$f(x) \approx 0.5x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right] \right)$$

Graph

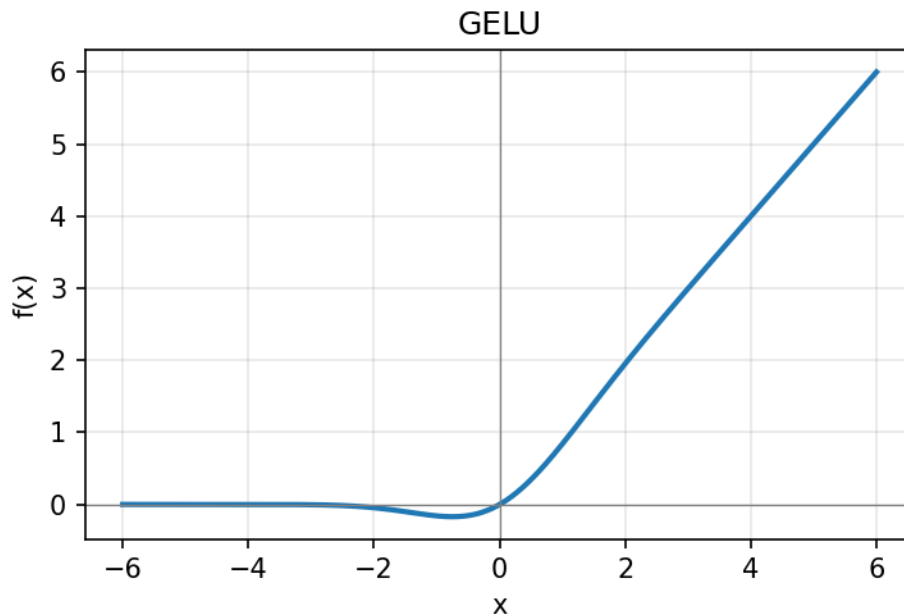


Figure 14: GELU

Code

```
import numpy as np
from scipy.stats import norm

def gelu_exact(x):
    return x * norm.cdf(x)
```

```
def gelu_approx(x):
    return 0.5 * x * (1 + np.tanh(np.sqrt(2/np.pi) * (x + 0.044715 * x**3)))

# PyTorch
import torch.nn as nn
y = nn.GELU()(torch.tensor([-2.0, 0.0, 2.0])) # exact (erf-based)
y2 = nn.GELU(approximate='tanh')(torch.tensor([-2.0, 0.0, 2.0])) # tanh approx
```

How to use

```
import torch.nn as nn
# Standard Transformer feed-forward block
class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model)
        )
    def forward(self, x):
        return self.net(x)
```

Where it's used

- **BERT, GPT-2/3/4, RoBERTa**, and most modern Transformer-based language models (feed-forward sublayers)
- Vision Transformers (ViT)
- Many state-of-the-art NLP and multimodal architectures

Why used

- Empirically performs better than ReLU in Transformers — smoother, retains small amounts of negative information (doesn't hard-zero negative inputs)
- Probabilistic motivation connects well with dropout-style regularization theory

Disadvantages

- More computationally expensive (Gaussian CDF / tanh-based approximation)
- Slightly more complex to implement from scratch correctly (numerical approximation needed for the exact erf form)

16. Hard Sigmoid

Explanation

Hard Sigmoid is a cheap, piecewise-linear approximation of the Sigmoid function. Instead of computing an expensive exponential, it uses simple linear segments clipped between 0 and 1 — much faster for embedded/mobile inference.

Math

A common form:

$$f(x) = \text{clip}(0.2x + 0.5, 0, 1)$$

(Other equivalent piecewise forms also used, e.g. $\text{clip}((x+3)/6, 0, 1)$ — implementations vary slightly between frameworks.)

Graph

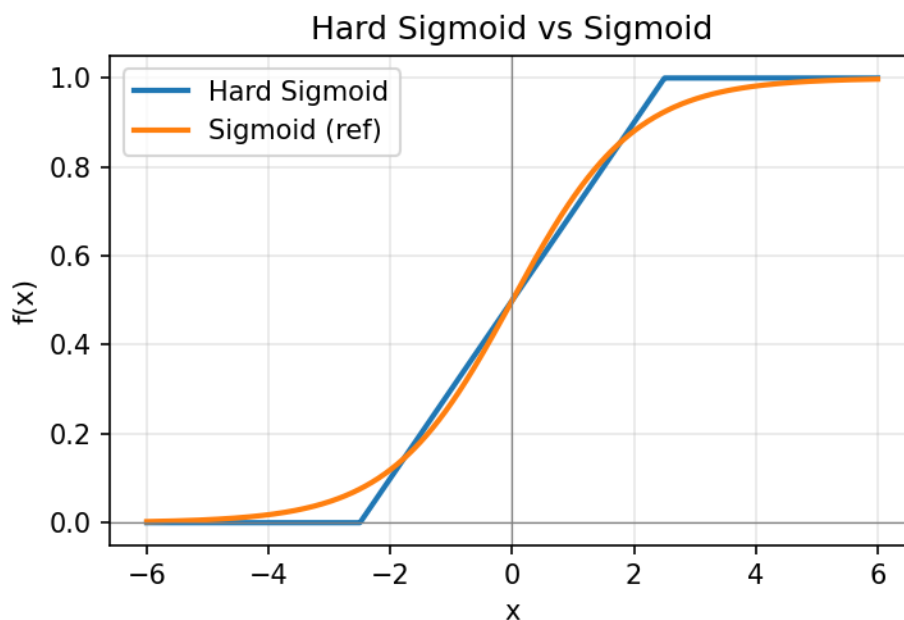


Figure 15: Hard Sigmoid

Code

```
import numpy as np

def hard_sigmoid(x):
    return np.clip(0.2 * x + 0.5, 0, 1)

# PyTorch
import torch.nn as nn
y = nn.Hardsigmoid()(torch.tensor([-4.0, 0.0, 4.0]))
```

How to use

```
import torch.nn as nn
# Common in lightweight mobile architectures
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.Hardsigmoid(),
    nn.Linear(64, 1)
)
```

Where it's used

- **MobileNetV3** (Squeeze-and-Excitation gating uses Hard Sigmoid/Hard Swish for speed)
- Embedded / edge / mobile deployment models where exponentials are expensive

Why used

- Much faster to compute (no `exp`) — important on CPUs/microcontrollers without fast exponential hardware
- Close enough approximation to Sigmoid for many practical gating purposes

Disadvantages

- Less smooth (piecewise linear => kinks at the clip boundaries, second derivative discontinuities)
 - Slight loss of accuracy/expressiveness compared to true Sigmoid
 - Gradient is exactly 0 outside the linear region, so neurons can saturate completely there (similar issue to ReLU's dying neuron problem)
-

17. Hard Tanh

Explanation

Just like Hard Sigmoid approximates Sigmoid, Hard Tanh is a cheap piecewise-linear approximation of Tanh — linear in the middle, clipped flat at -1 and +1 outside a range.

Math

$$f(x) = \begin{cases} -1 & x < -1 \\ x & -1 \leq x \leq 1 \\ 1 & x > 1 \end{cases}$$

Graph

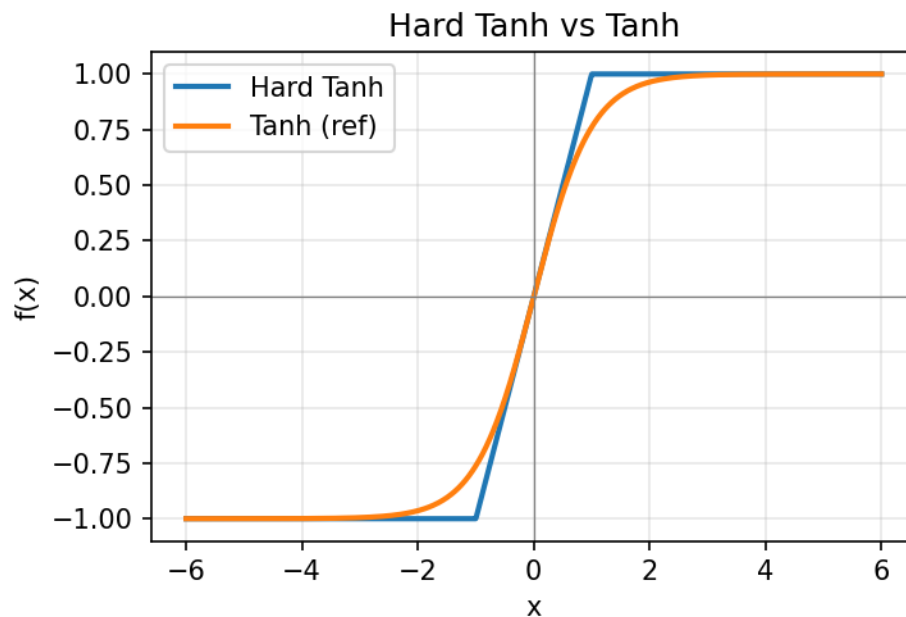


Figure 16: Hard Tanh

Code

```
import numpy as np

def hard_tanh(x):
    return np.clip(x, -1, 1)

# PyTorch
import torch.nn as nn
y = nn.Hardtanh()(torch.tensor([-2.0, 0.0, 2.0]))
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.Hardtanh(),
    nn.Linear(64, 1)
)
```

Where it's used

- Quantized / low-precision neural networks (mobile NLP/speech models)
- Some early RNN implementations needing a cheaper Tanh substitute
- Binary/ternary neural network research

Why used

- Extremely cheap to compute (just a clip operation)
- Useful when targeting hardware with limited support for transcendental functions (exp, tanh)

Disadvantages

- Flat regions outside $[-1, 1]$ give exactly zero gradient there $\Rightarrow$ neurons can get stuck (similar to dying ReLU)
 - Less smooth and less expressive than true Tanh
 - Rarely used in large-scale modern architectures
-

18. Maxout

Explanation

Maxout is different from the others — rather than transforming the output of a single linear unit, a Maxout unit takes the **maximum across several separate linear pieces**, each with its own weights and biases. This makes Maxout a generalization that can itself learn to approximate ReLU, Leaky ReLU, or even convex piecewise-linear functions.

Math

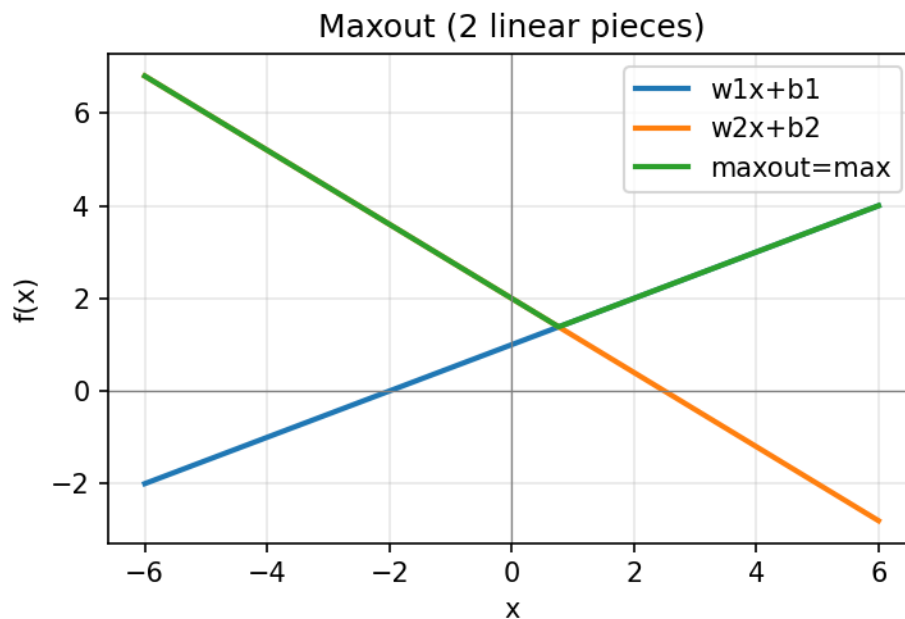
For k linear pieces:

$$f(x) = \max_{i=1,\dots,k} (w_i^T x + b_i)$$

For the simple 2-piece illustration:

$$f(x) = \max(w_1x + b_1, w_2x + b_2)$$

Graph



(Two linear pieces, and the resulting Maxout = element-wise max of the two lines)

Code

```
import numpy as np
import torch
import torch.nn as nn

class Maxout(nn.Module):
    def __init__(self, in_features, out_features, num_pieces=4):
```

```

    super().__init__()
    self.num_pieces = num_pieces
    self.out_features = out_features
    # one big linear layer producing num_pieces * out_features outputs
    self.linear = nn.Linear(in_features, out_features * num_pieces)

    def forward(self, x):
        out = self.linear(x)
        out = out.view(*out.shape[:-1], self.out_features, self.num_pieces)
        return out.max(dim=-1)[0]

layer = Maxout(in_features=10, out_features=5, num_pieces=4)
y = layer(torch.randn(2, 10))

```

How to use

```

model = nn.Sequential(
    Maxout(in_features=784, out_features=256, num_pieces=4),
    Maxout(in_features=256, out_features=10, num_pieces=4)
)

```

Where it's used

- Originally proposed alongside **Dropout** by Goodfellow et al. (2013) to work especially well together
- Some older CNN benchmark models (e.g., early MNIST/CIFAR research models)

Why used

- It's a **universal function approximator on its own** — a Maxout layer with enough pieces can approximate any convex piecewise-linear function, including ReLU, Leaky ReLU, and absolute value
- Works very well combined with Dropout, since it doesn't have ReLU's “dying neuron” issue (always has nonzero gradient through the winning linear piece)

Disadvantages

- Multiplies the number of parameters by k (the number of linear pieces) — significantly more expensive than ReLU
 - Rarely used in modern large-scale models due to the parameter/compute overhead
 - More complex to implement and tune (extra hyperparameter k)
-

19. ReLU6

Explanation

ReLU6 is simply ReLU but additionally capped/clipped at 6. It was designed for **quantized / fixed-point** neural networks, where bounding the output range makes converting activations to low-precision integers (like 8-bit) much more numerically stable and predictable.

Math

$$f(x) = \min(\max(0, x), 6)$$

Graph

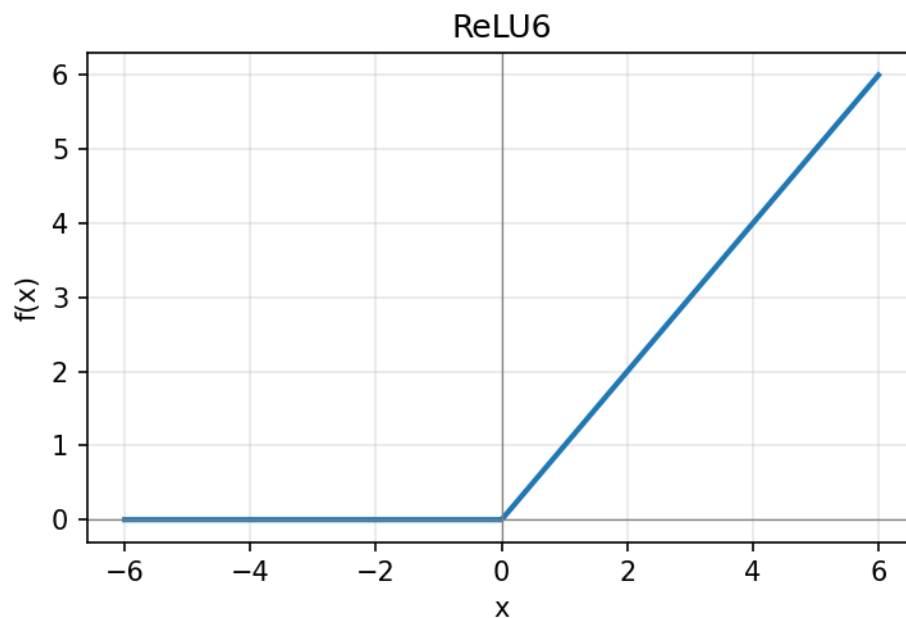


Figure 17: ReLU6

Code

```
import numpy as np

def relu6(x):
    return np.clip(x, 0, 6)

# PyTorch
import torch.nn as nn
y = nn.ReLU6()(torch.tensor([-2.0, 3.0, 8.0]))
```

How to use

```
import torch.nn as nn
model = nn.Sequential(
    nn.Conv2d(3, 32, 3),
    nn.ReLU6(),      # bounded activation, good for INT8 quantization
    nn.Conv2d(32, 64, 3),
    nn.ReLU6()
)
```

Where it's used

- **MobileNetV1 / MobileNetV2** (Google's efficient mobile CNN architectures)
- Quantization-aware training pipelines (TensorFlow Lite, mobile/edge deployment)

Why used

- Bounded output range (0 to 6) makes fixed-point/INT8 quantization much more accurate and stable than unbounded ReLU
- Still retains ReLU's cheap computation and sparsity benefits

Disadvantages

- The upper cap at 6 can clip useful large activation values, potentially losing information in some layers
 - The “6” is a somewhat arbitrary empirical choice (chosen because it worked well in early mobile quantization experiments)
 - Less commonly needed outside of quantized/mobile deployment scenarios
-

20. Master Comparison Table

Function	Range	Zero-Centered?	Smooth?	Monotonic?	Computational Cost	Common Use
Linear	$(-\infty, \infty)$	Yes	Yes	Yes	Lowest	Regression output
Binary Step	$\{0, 1\}$	No	No	Yes	Lowest	Historical perceptrons
Sigmoid	$(0, 1)$	No	Yes	Yes	Medium	Binary output, gates
Tanh	$(-1, 1)$	Yes	Yes	Yes	Medium	RNN/LSTM hidden states
ReLU	$[0, \infty)$	No	No (kink at 0)	Yes	Lowest	CNN/MLP hidden layers
Leaky ReLU	$(-\infty, \infty)$	No	No	Yes	Low	GANs, CNNs
PReLU	$(-\infty, \infty)$	No	No	Yes	Low+	Deep CNNs
ELU	$(-\alpha, \infty)$	Near	Yes	Yes	Medium	Faster-converging CNNs
SELU	$(-\lambda\alpha, \infty)$	Near	Yes	Yes	Medium	Self-normalizing MLPs
Softmax	$(0, 1)$ sums to 1	No	Yes	N/A (vector)	Medium	Multi-class output, attention
Softplus	$(0, \infty)$	No	Yes	Yes	Medium	Positive-output constraints
Softsign	$(-1, 1)$	Yes	Yes	Yes	Low	Alt. to Tanh
Swish/SiLU	$\approx (-0.28, \infty)$	No	Yes	No	Medium	EfficientNet, YOLO
Mish	$\approx (-0.31, \infty)$	No	Yes	No	Medium-High	YOLOv4
GELU	$\approx (-0.17, \infty)$	No	Yes	No	Medium-High	Transformers (BERT, GPT)
Hard Sigmoid	$[0, 1]$	No	No	Yes	Lowest	Mobile gating (MobileNetV3)
Hard Tanh	$[-1, 1]$	Yes	No	Yes	Lowest	Quantized/low-precision nets

Function	Range	Zero-Centered?	Smooth?	Monotonic?	Computational Cost	Common Use
Maxout	$(-\infty, \infty)$	No	No	No	High ($k \times \text{params}$)	Research, Dropout pairing
ReLU6	$[0, 6]$	No	No	Yes	Lowest	MobileNet, quantized nets

21. How to Choose an Activation Function — Practical Guide

1. **Hidden layers, general-purpose CNN/MLP** → Start with **ReLU**. It's the default for good reason: cheap, fast, works well in most cases.
2. **ReLU dying / underperforming** → Try **Leaky ReLU** or **PReLU**.
3. **Transformer-based model (NLP/Vision Transformers)** → Use **GELU** (the de facto standard for BERT/GPT-style models).
4. **Mobile / edge deployment, need quantization** → Use **ReLU6** or **Hard Sigmoid / Hard Swish**.
5. **RNN / LSTM internal gates** → **Sigmoid** (gates) and **Tanh** (cell/candidate state) — this is baked into the LSTM/GRU equations themselves.
6. **Final output layer:**
 - Regression → **Linear**
 - Binary classification → **Sigmoid**
 - Multi-class classification → **Softmax**
 - Multi-label classification → **Sigmoid** (applied independently per class)
7. **Want best possible accuracy and can afford compute** → Try **Swish/SiLU** or **Mish** (common in modern CNN object detectors).
8. **Deep plain MLP, no BatchNorm, fully-connected only** → Consider **SELU** with LeCun initialization.
9. **Predicting a value that must be strictly positive** (e.g., variance, scale) → **Softplus**.

General rules of thumb

- Prefer **non-saturating** functions (ReLU family, GELU, Swish) in deep networks — they suffer far less from vanishing gradients than Sigmoid/Tanh.
 - Sigmoid/Tanh are mostly relegated today to **gating mechanisms** (LSTM/GRU) and **output layers**, not deep hidden stacks.
 - When in doubt for a brand-new architecture: **ReLU first, GELU/Swish second** — these cover the vast majority of real-world use cases.
 - Always pair activation choice with proper **weight initialization** (e.g., He initialization for ReLU family, Xavier/Glorot for Tanh/Sigmoid, LeCun for SELU).
-

22. The Vanishing & Exploding Gradient Problem (Why Activation Choice Matters So Much)

During backpropagation, gradients are computed via the chain rule, multiplying many layers' derivatives together:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a_n} \cdot \frac{\partial a_n}{\partial a_{n-1}} \dots \frac{\partial a_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1}$$

- If each $\frac{\partial a_i}{\partial a_{i-1}}$ term (which depends on the activation function's derivative) is **small** (e.g., Sigmoid's derivative maxes out at 0.25), the product of many small numbers shrinks exponentially toward zero as the network gets deeper — this is the **vanishing gradient problem**. Early layers stop learning.
- If those terms are consistently **greater than 1**, the product can grow exponentially — the **exploding gradient problem** — causing unstable, huge weight updates.

This is exactly why: - Sigmoid/Tanh struggle in very deep networks (their derivatives are ≤ 1 , often much less, almost everywhere) - ReLU (derivative = 1 for all positive inputs) helps gradients flow much better through many layers - Modern smooth alternatives (GELU, Swish, Mish) try to keep gradients flowing well while still being smooth and slightly non-monotonic, which empirically helps optimization

23. Implementing a Tiny Neural Network From Scratch (Putting It Together)

This shows how an activation function integrates into the actual forward and backward pass of a real network — using ReLU as the example.

```
import numpy as np

class SimpleNN:
    def __init__(self, input_dim, hidden_dim, output_dim, lr=0.01):
        self.W1 = np.random.randn(input_dim, hidden_dim) * np.sqrt(2/input_dim) # He init
        self.b1 = np.zeros(hidden_dim)
        self.W2 = np.random.randn(hidden_dim, output_dim) * np.sqrt(2/hidden_dim)
        self.b2 = np.zeros(output_dim)
        self.lr = lr

    def relu(self, x):
        return np.maximum(0, x)

    def relu_deriv(self, x):
        return (x > 0).astype(float)

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def forward(self, X):
        self.z1 = X @ self.W1 + self.b1
        self.a1 = self.relu(self.z1) # <-- activation function used here
        self.z2 = self.a1 @ self.W2 + self.b2
        self.a2 = self.sigmoid(self.z2) # <-- output activation
        return self.a2

    def backward(self, X, y):
        m = X.shape[0]
        dz2 = self.a2 - y # derivative of BCE + sigmoid combined
        dW2 = self.a1.T @ dz2 / m
        db2 = dz2.mean(axis=0)

        da1 = dz2 @ self.W2.T
        dz1 = da1 * self.relu_deriv(self.z1) # <-- chain rule through ReLU derivative
        dW1 = X.T @ dz1 / m
        db1 = dz1.mean(axis=0)

        self.W2 -= self.lr * dW2
        self.b2 -= self.lr * db2
        self.W1 -= self.lr * dW1
        self.b1 -= self.lr * db1
```

```

def train(self, X, y, epochs=1000):
    for epoch in range(epochs):
        self.forward(X)
        self.backward(X, y)

# Example usage on a tiny dataset
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]]) # XOR problem - needs non-linearity!
net = SimpleNN(2, 8, 1)
net.train(X, y, epochs=5000)
print(net.forward(X))

```

Note the key point: solving XOR is *impossible* with a purely linear model — it is the classic textbook example proving why non-linear activation functions are mandatory for anything beyond trivial linearly-separable problems.

24. Summary Cheat Sheet (One-Liners)

- **Linear** — no non-linearity; only for regression output layers.
 - **Binary Step** — historical; on/off; not usable with gradient descent.
 - **Sigmoid** — squashes to (0,1); probabilities; vanishing gradient risk; used in gates.
 - **Tanh** — squashes to (-1,1); zero-centered version of sigmoid; used in RNN/LSTM.
 - **ReLU** — $\max(0, x)$; cheap, default choice; can “die” on negative inputs.
 - **Leaky ReLU** — ReLU with small negative slope; fixes dying neurons partially.
 - **PReLU** — Leaky ReLU with learnable slope.
 - **ELU** — smooth negative saturation; faster convergence than ReLU sometimes.
 - **SELU** — scaled ELU; enables self-normalizing deep MLPs.
 - **Softmax** — vector-wise; converts logits to a probability distribution; multi-class output & attention.
 - **Softplus** — smooth ReLU approximation; always positive output; derivative = sigmoid.
 - **Softsign** — cheap Tanh alternative using division instead of exponentials.
 - **Swish/SiLU** — x times $\text{sigmoid}(x)$; smooth, non-monotonic; used in EfficientNet.
 - **Mish** — x times $\tanh(\text{softplus}(x))$; smooth; used in YOLOv4.
 - **GELU** — x times Gaussian CDF(x); standard activation in Transformers (BERT/GPT).
 - **Hard Sigmoid** — cheap piecewise-linear sigmoid approximation; mobile-friendly.
 - **Hard Tanh** — cheap piecewise-linear tanh approximation; quantization-friendly.
 - **Maxout** — max over several learned linear pieces; very expressive, but parameter-heavy.
 - **ReLU6** — ReLU capped at 6; designed for quantized mobile networks.
-

25. References & Further Reading

- Nair, V., & Hinton, G. E. (2010). *Rectified Linear Units Improve Restricted Boltzmann Machines*. ICML.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification* (PReLU paper).
- Clevert, D-A., Unterthiner, T., & Hochreiter, S. (2015). *Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)*.
- Klambauer, G., Unterthiner, T., Mayr, A., & Hochreiter, S. (2017). *Self-Normalizing Neural Networks* (SELU paper).
- Goodfellow, I., et al. (2013). *Maxout Networks*. ICML.
- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). *Searching for Activation Functions* (Swish paper).
- Misra, D. (2019). *Mish: A Self Regularized Non-Monotonic Activation Function*.
- Hendrycks, D., & Gimpel, K. (2016). *Gaussian Error Linear Units (GELUs)*.
- Howard, A., et al. (2019). *Searching for MobileNetV3* (Hard Swish, Hard Sigmoid).
- Vaswani, A., et al. (2017). *Attention Is All You Need* (Softmax in self-attention, GELU/ReLU FFN).

End of notes.